

行业 Agent 架构方案设计

版本说明

版本号	更新说明	更新时间	备注
v0.0.2	1. 增加链路架构图	2026 年 4 月 22 日	
v0.0.1	1. 第一版技术架构图，包含完整模块	2026 年 4 月 13 日	

1. 背景与目标

1.1 背景介绍

随着 AI 大模型能力和 AI 智能体的快速成熟，越来越多的行业系统开始引入智能能力以提升业务处理效率。然而，当前主流的 Agent 产品或平台大多以通用任务助手形式存在，缺乏针对行业正式业务链路的严谨组织方式，难以满足以下需求：

- 正式业务事项的完整追溯与审计
- 高风险操作的人工确认与权限管控
- 跨行业能力复用与快速切换
- 与既有领域系统的低侵入集成
- 结合模式以传统软件+AI

本架构方案在智能业务框架（业务组织层）的基础上，定义行业 Agent 的完整技术实现方式，覆盖图书馆、烟草、水务三个典型行业场景，形成可落地、可复用、可扩展的AI架构基座。

1.2 设计目标

鉴于当前行业情况，本设计方案主要实现如下目标：

- 正式处理链闭环，Agent 处理正式业务服务/事项时，覆盖进入、识别、执行、确认、留痕的完整链路，不依赖线下补充

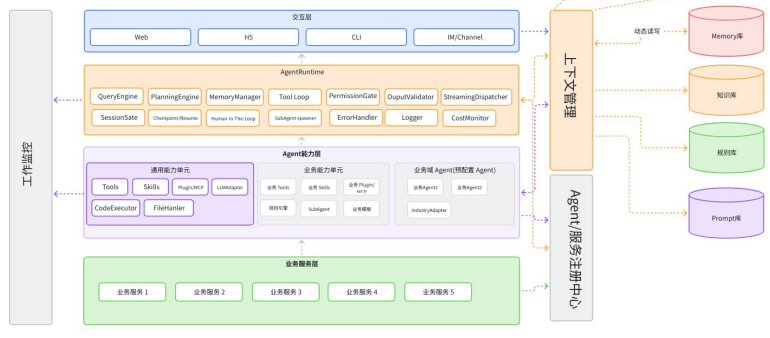
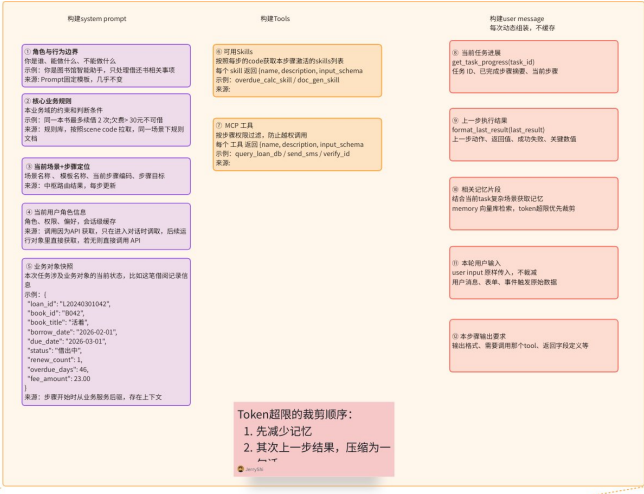
- 多行业快速切换，通过 Industry Adapter + 业务能力单元替换，实现跨行业复用，上层 Runtime 不感知行业差异
- 人机边界清晰，高风险操作必须经由 Permission Gate 和 Human-in-the-Loop 控制，自动推进不越界
- 上下文全程受控，Context 贯穿全链路，确保建议生成、确认推进、正式执行的信息一致性
- 低侵入集成，框架建立在既有领域系统之上，不重写底层规则，不替代领域服务
- 可审计可追溯，关键处理节点形成完整审计链路，支持复查、合规和复盘

1.3 设计原则

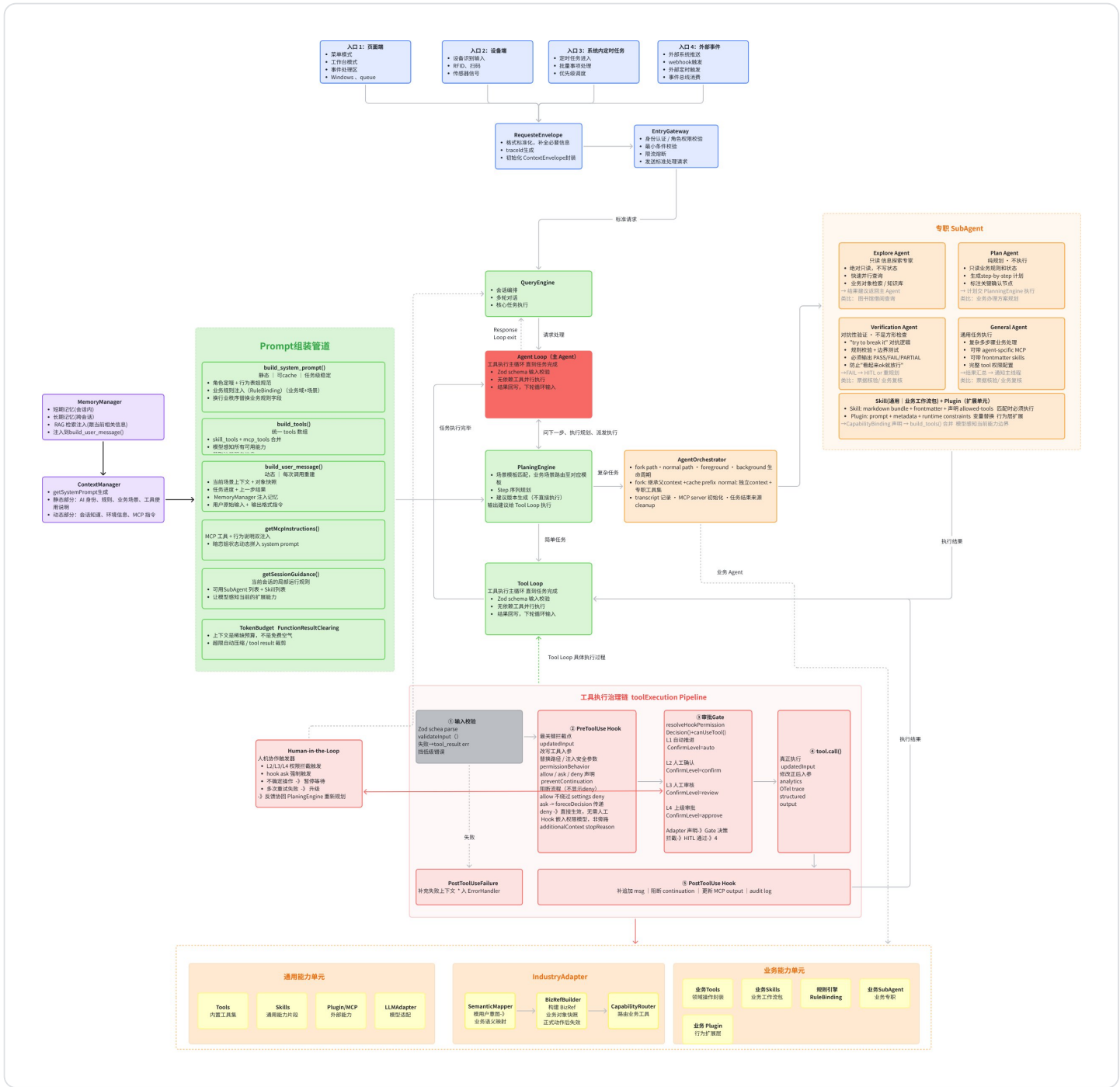
- 正式处理链原则：Agent 进入正式业务处理链，系统承接建议、确认、执行与追溯
- 场景优先原则：处理组织以场景为起点，不以页面或接口为起点
- 人工边界原则：高风险动作配置明确确认机制，自动推进不得越界
- 状态一致原则：运行状态与正式业务状态保持一致，不允许状态断裂
- 留痕闭环原则：处理过程形成完整运行链路，支撑治理与责任校验
- 低侵入接入原则：在既有业务系统之上增加统一处理组织能力，不替代领域服务
- 行业可替换原则：业务能力单元与 Runtime 解耦，切换行业只替换能力层，不改变 Runtime 主干

2. 架构设计

2.1 技术架构



2.2 链路架构图



支持用户修改/创建已有编排的快路径业务规则或工作流。

2.3 模块说明

AgentRuntime 层

1. **Query Engine** — 用户输入的第一道处理管道。负责意图识别（判断用户要做什么）、槽位提取（抽取结构化参数）、Query 改写扩写（补全模糊表达、多路扩写提升召回）、上下文融合（把当前输入和历史上下文拼合）。输出是结构化的任务描述对象，交给 Planning Engine。

2. **Planning Engine** — 任务拆解模块。判断当前任务是否需要分步执行，如果需要则生成有序的执行计划（对应 Claude Code 的 `TodoWrite`）。系统在每次工具调用后会当前计划状态注入提示，防止模型在长任务中迷失。
3. **Memory Manager** — 记忆调度决策器。不自己存储数据，而是调用上下文管理的接口，决策：当前步骤需不需要查长期记忆、查哪类记忆、任务结束后哪些内容值得持久化写回 Memory 库。
4. **Tool Loop** — Agent 的核心执行引擎。对应 Claude Code 的主循环 `n0`：模型决定调用工具 → 执行工具 → 结果回传模型 → 继续下一步，直到模型不再调用工具为止。维护单一扁平的消息历史。
5. **Permission Gate** — 操作权限拦截点。每个工具执行前校验权限，高风险操作（删除、提交、支付）触发显式确认，防止 Agent 越权执行破坏性操作。对应 Claude Code 的三层权限模型：Trust 建立 → Permission 校验 → User 确认。
6. **Output Validator** — 输出质量校验。检查模型输出的格式合规性、内容安全性，不符合预期时自动触发重试，避免把错误结果直接返回给用户。
7. **Streaming Dispatcher** — 流式输出管理。对应 Claude Code 的 `StreamGen`，负责 token 级别的流式推送，让用户在模型还在生成时就能看到内容，提升响应体验。同时管理对应 `h2A` 的中断插入机制，支持用户在任务执行中途注入新指令。
8. **Session State** — 会话状态管理。维护当前会话的完整状态，包括多轮对话历史、当前执行进度、用户偏好设置，确保跨多轮交互的连贯性。
9. **Checkpoint / Resume** — 断点续跑机制。对应 Claude Code 的长任务中断恢复能力。将任务执行状态定期持久化，网络中断或用户主动暂停后可以从断点继续，不需要从头重跑。
10. **Human-in-the-Loop** — 人工介入触发器。当 Agent 遇到不确定的操作、Permission Gate 拦截到高风险行为、或 Output Validator 多次重试失败时，暂停执行并等待人工确认后继续。
11. **SubAgent Spawner** — 子 Agent 派生器。对应 Claude Code 的 `dispatch_agent`（I2A/Task Agent）。当主任务需要并行探索或分支处理时，派生独立的子 Agent 执行子任务。有深度限制，子 Agent 不能再派生自己的子 Agent，防止递归失控。
12. **Error Handler** — 异常处理。捕获工具调用失败、模型超时、外部服务报错等异常，决策是重试、降级还是终止任务，并把错误信息整理后注入上下文供模型感知。
13. **Logger** — 执行日志。记录每一步工具调用、模型决策、上下文变化的完整链路，供工作监控层消费，用于调试、审计和性能分析。
14. **Cost Monitor** — 成本监控。对应 Claude Code 的 `/cost` 命令能力。实时追踪每次会话的 token 消耗和估算费用，支持设置预算上限，超限时触发告警或降级到轻量模型。

Agent 能力层 — 通用能力单元

1. **Tools** — Agent 的行动手段。包含所有内置工具：文件读写、Shell 命令执行、Web 搜索、URL 抓取、知识检索等。对应 Claude Code 的工具集（View、Edit、Bash、GrepTool、WebFetch 等）。RAG 检索也是 Tools 的一个具体实现，不单独抽层。

代码块

1 Tools (基于 Claude Code 官方工具集)

2 Claude Code 的内置工具分为五大类，权限要求各不相同：

3 文件操作类：

4 Read – 读取文件内容，无需权限

5 Write – 创建或整体覆盖文件，需要权限

6 Edit – 对特定文件做局部精准修改（Diff 模式），需要权限

7 NotebookEdit – 修改 Jupyter Notebook 的 cell，需要权限

8 搜索类：

9 Glob – 按模式匹配查找文件，无需权限

10 Grep – 正则搜索文件内容，无需权限

11 LSP – 代码智能：跳转定义、查找引用、类型报错，无需权限

12 执行类：

13 Bash – 执行 Shell 命令，持久化工作目录，需要权限

14 PowerShell – Windows 环境下执行 PowerShell，需要权限

15 Web 类：

16 WebSearch – 执行网页搜索，需要权限

17 WebFetch – 抓取指定 URL 内容，需要权限

18 编排与规划类：

19 Agent – 派生子 Agent，拥有独立上下文窗口，无需权限

20 AskUserQuestion – 向用户提多选问题以澄清需求，无需权限

21 EnterPlanMode / ExitPlanMode – 切入/退出规划模式，设计方案后再执行

22 TodoWrite – 管理会话任务清单（非交互模式用）

23 TaskCreate / TaskGet / TaskList / TaskUpdate / TaskStop – 任务全生命周期管理

24 SendMessage – 向 Agent 团队成员发消息（实验性）[待定]

25 TeamCreate / TeamDelete – 创建/解散多 Agent 团队（实验性）【待定】

26 定时任务类：

27 CronCreate – 在当前会话内创建定时或一次性任务

28 CronDelete / CronList – 管理定时任务

29 MCP 集成类：

30 ListMcpResourcesTool – 列出已连接 MCP 服务器暴露的资源

31 ReadMcpResourceTool – 按 URI 读取指定 MCP 资源

32 ToolSearch – 搜索并加载延迟工具（tool search 模式下）

2. **Skills** — 可复用的通用能力片段。与行业无关的执行序列，比如文本摘要、多语言翻译、格式转换、数据提取。可以理解为对多个 Tools 的高层封装，供上层直接调用。
3. **Plugin / MCP** — 外部能力标准化接入。基于 MCP（Model Context Protocol）协议，把第三方服务、外部 API、专用工具统一封装成 Agent 可调用的标准工具，扩展 Tools 的边界而不修改核心。
4. **LLM Adapter** — 模型适配层。屏蔽不同 LLM 厂商的 API 差异（Anthropic、OpenAI、本地模型等），支持按任务复杂度路由到不同规格模型——简单意图识别用轻量模型，复杂规划用大模型，控制成本同时保留能力上限。
5. **Code Executor** — 代码沙箱执行。在隔离环境中安全运行动态生成的代码，结果回传给 Tool Loop。适用于数据分析、报表生成、自动化脚本等需要执行代码的业务场景。

6. **File Handler** — 文件操作管理。处理复杂的文件读写需求，支持 Diff 对比、版本追踪、批量操作。对应 Claude Code 的 Edit/Write 工具，但在企业场景下增加了版本管理能力。
-

Agent 能力层 — 业务能力单元

1. **业务 Tools** — 行业专属工具。与通用 Tools 结构相同，但内容是行业定制的：图书馆场景有"查馆藏"、"办理借阅"；烟草场景有"查库存"、"提交订单"；水务场景有"读取水表"、"报修工单"。切换行业时只替换这里的工具集。
 2. **业务 Skills** — 行业工作流片段。把多个业务 Tools 组合成完整的业务流程，比如"图书采编流程"（选书→报批→采购→编目）、"烟草盘点流程"（扫码→核对→差异上报）。比单个 Tool 粒度更大，比预装配 Agent 粒度更小。
 3. **规则引擎** — 行业业务规则管理。从规则库加载当前行业的合规要求、审批约束、操作限制，在 Agent 执行前校验行为是否符合业务规则。不同行业挂载不同规则集，切换行业时规则自动切换。
 4. **业务 Plugin / MCP** — 行业专属外部服务接入。对接行业特定的第三方系统：图书馆的 ILMS、烟草的 ERP、水务的 SCADA，通过 MCP 协议标准化后暴露给 Agent 调用。
 5. **SubAgent** — 聚合业务能力的执行单元。把多个业务 Tools 和业务 Skills 组合成一个可独立执行的业务能力包，由 Runtime 的 SubAgent Spawner 派生调度。比如"采编 SubAgent"封装了采编相关的所有工具和流程，主 Agent 只需要派发任务，不需要知道内部细节。
 6. **业务模版** — 行业提示词、配置模板、固定工作流程。包含行业专属的系统提示词、任务模板、输出格式规范。当系统识别到某些固定场景时不需要通过前者那么多流程，可以直接使用业务模版快速完成任务。
 7. **Industry Adapter** — 行业切换的核心抽象。把不同行业的业务语义统一映射成 Runtime 能理解的标准动作，屏蔽行业差异。切换行业时只换 Adapter + 业务能力单元，上层 Runtime 和通用能力完全不感知行业变化。这是整个架构实现"与行业脱离"目标的关键模块。
-

数据存储库

1. **Memory 库** — 动态读写的记忆存储。保存用户偏好、历史任务结果、Agent 学到的业务知识。有短期记忆（会话级）和长期记忆（跨会话）之分。对应 Claude Code 的 memory.md 设计，优先用可读可审计的格式存储，而非不透明的向量库。
2. **知识库** — 静态领域知识。行业的基础知识、产品目录、操作手册等，以只读为主，通过 RAG 检索注入上下文。
3. **规则库** — 业务规则和约束。审批流程、合规要求、操作权限定义，由规则引擎加载执行前校验。相对稳定，变更需要走版本管理。
4. **Prompt 库** — 提示词模板库。系统提示词、任务模板、Few-shot 示例，由 Query Engine 在构建初始上下文时加载，决定 Agent 的行为风格和任务理解框架。

1. Session 管理组件
2. 上下文结构及实现

2.4 详细模块设计

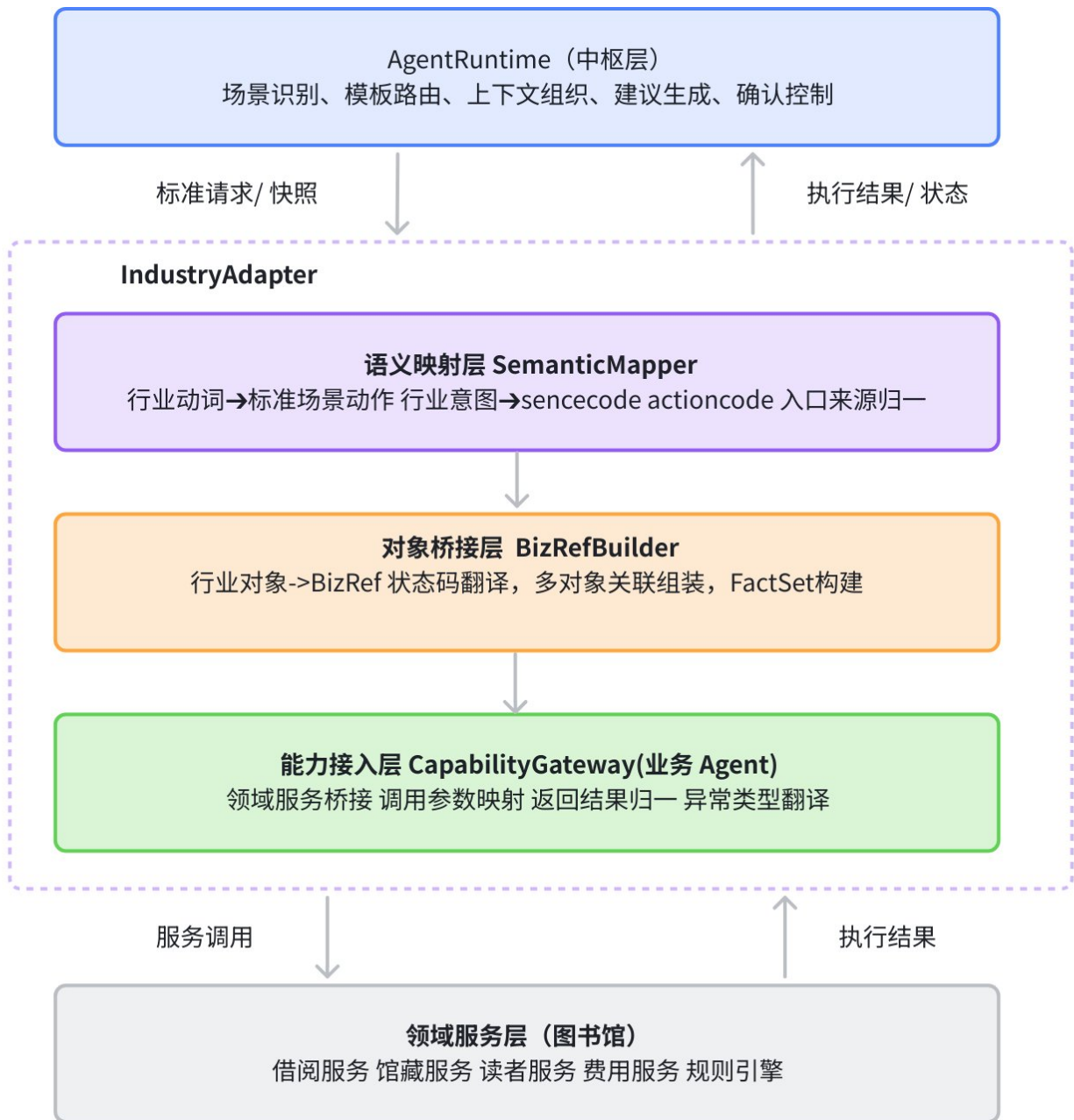
2.4.1 Industry Adapter

需要结合具体行业比如图书馆业务场景流转逻辑进行详细设计

Industry Adapter是适配层中行业化实现层，本质上需要完成三件事：

1. 语义映射：图书馆业务语义(这里以此行业为例，可平行扩展到其他行业)，映射到 Runtime 标准动作
2. 对象桥接：图书馆业务对象（读者/馆员/图书等）映射到框架运行对象
3. 能力接入：图书馆领域服务，映射到框架可调用的标准能力接口或组装好的 场景业务能力

基于这个目标，Industry Adapter初步设计结构如下：



2.4.1.1 语义映射层

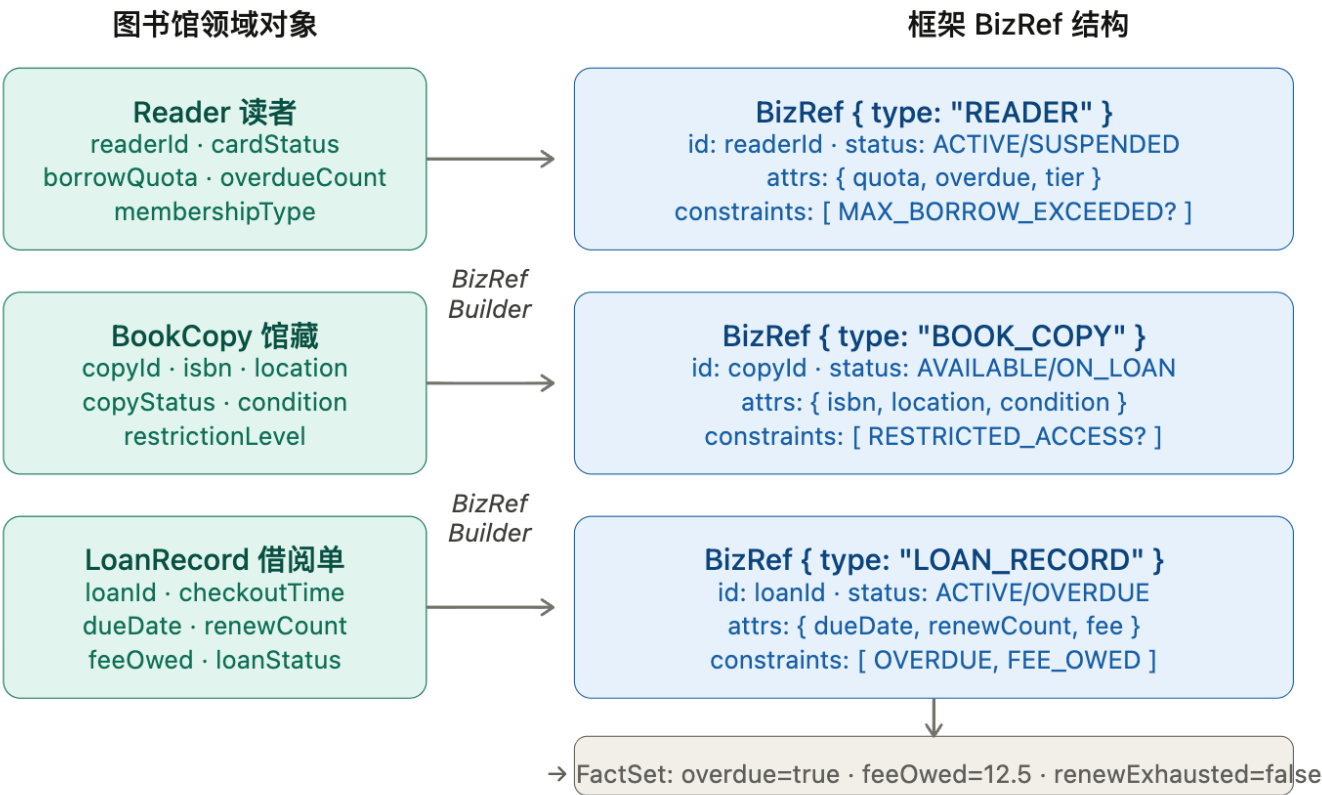
图书馆行业有自己的动词体系，AgentRuntime 有自己的标准动作类型，SemanticMapper 负责两者之间的双向翻译。示例如下

图书馆语义 → 标准动作映射表：

图书馆业务动词	标准 sceneCode	标准 actionCode	路径类型
扫码借书	CIRCULATION_CHECKOUT	ACTION_INIT	快路径
柜台归还	CIRCULATION_RETURN	ACTION_INIT	快路径
自助续期	CIRCULATION_RENEW	ACTION_INIT	快路径
预约取书	CIRCULATION_RESERVE_FULFILL	ACTION_INIT	快路径
状态争议处理	DISPUTE_STATUS	ACTION_INIT	重路径
费用争议核查	DISPUTE_FEE	ACTION_INIT	重路径
特殊授权审核	AUTH_SPECIAL	ACTION_INIT	重路径
异常工单处理	INCIDENT_HANDLE	ACTION_INIT	重路径

2.4.1.2 对象桥接层(BizRefbuilder)

AgentRuntime 有自己的上下文对象（即整个流转的数据结构内容）BizRef（临时定义名称），图书馆系统有自己的对象模型。BizRefbuilder 负责将图书馆领域对象包装为框架可识别的引用结构。示例如下。



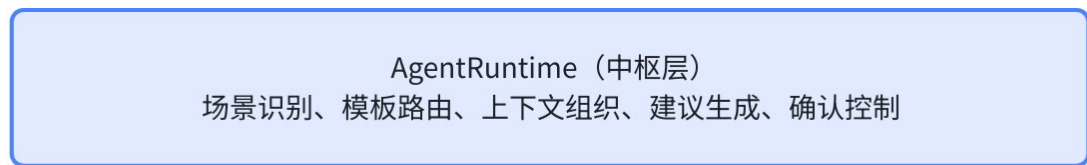
状态码翻译示例（图书馆 → 框架）：

图书馆状态码	框架通用状态	约束标志
CARD_FROZEN	SUBJECT_SUSPENDED	ACCESS_BLOCKED
COPY_ON_LOAN	OBJECT_UNAVAILABLE	—
LOAN_OVERDUE	RECORD_EXCEPTION	OVERDUE, FEE_OWED
RENEW_LIMIT_REACHED	OPERATION_EXHAUSTED	RENEW_BLOCKED
RESTRICTED_COLLECTION	ACCESS_RESTRICTED	AUTH_REQUIRED

2.4.1.3 能力接入层 CapabilityGateway(业务 Agent)

这里只展示业务 Agent 对接过程

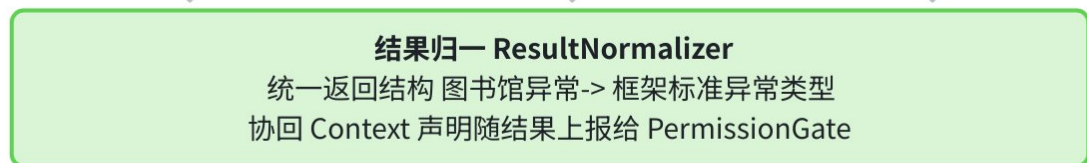
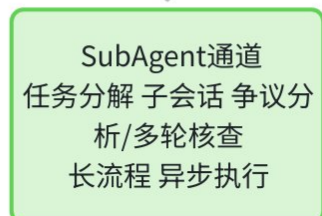
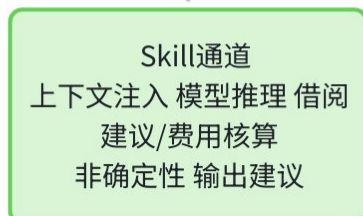
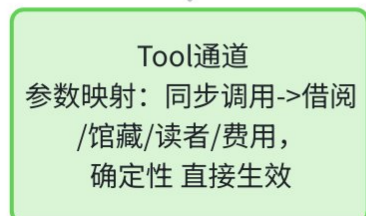
能力接入层实则是业务 Agent 这层，下游对接的是业务服务。整体设计如下。



标准请求/ 快照

执行结果/ 状态

IndustryAdapter



能力注册表示例: CHECKOUT_BOOK →
Tool • HUMAN_CONFIRM WAIVE_FEE →
Tool • SUPERIOR_APPROVAL

服务调用

执行结果



其他信息：

1. 当前 Claude Code Session 管理机制

